

SQL SERVER 2022 • AVANZADO

Optimización de accesos a disco e infraestructura de almacenamiento

Sesión 2 de 6 • Arquitectura de I/O, tempdb, Buffer Pool Extension, alineación y wait stats de almacenamiento

Por qué el disco sigue importando

- Una lectura lógica en memoria cuesta microsegundos; una física en disco, de 0,1 a decenas de milisegundos
- La memoria nunca es infinita: el buffer pool compite con la caché de planes y las concesiones de memoria
- El log de transacciones es síncrono: cada COMMIT espera a que el disco confirme la escritura
- tempdb es un recurso compartido por toda la instancia y es el origen de muchos problemas invisibles
- Regla del curso: primero descartar un mal plan; luego culpar al disco, nunca al revés

Idea clave

Muchos "problemas de disco" son en realidad consultas que leen cien veces más páginas de las necesarias. Un índice adecuado reduce las lecturas físicas mucho más barato que una cabina nueva.

01

Arquitectura de I/O en SQL Server

Páginas, extensiones, buffer pool, write-ahead logging y los procesos que escriben en segundo plano

El camino de una página

1

Petición

El operador del plan pide la página (archivo:página)

2

Buffer pool

¿Está en memoria?
Lectura lógica, microsegundos

3

Lectura física

Si no está: I/O asíncrono, la tarea espera en PAGEIOLATCH

4

Read-ahead

En escaneos, lee por delante bloques de hasta 512 KB

5

Modificación

La página queda "sucia" en memoria; se escribe más tarde

La unidad de almacenamiento es la página de 8 KB; ocho páginas contiguas forman una extensión de 64 KB.

Quién escribe en disco y cuándo

1

Log writer

Escribe el log de forma secuencial en cada COMMIT. Write-ahead logging: el log se escribe antes que la página de datos. Espera asociada: WRITELOG.

2

Checkpoint

Vuelca periódicamente páginas sucias a los ficheros de datos para acortar la recuperación. Con indirect checkpoint (por defecto, 60 s de objetivo) es más suave.

3

Lazy writer

Libera búferes cuando hay presión de memoria, escribiendo antes las páginas sucias. Actividad alta: falta memoria.

4

Tareas de usuario

Pueden escribir directamente en crecimientos de ficheros sin IFI, creación de índices o carga masiva mínimamente registrada.

Latencia real por fichero: sys.dm_io_virtual_file_stats

```
SELECT DB_NAME(vfs.database_id) AS bd, mf.type_desc, mf.physical_name,  
       vfs.num_of_reads, vfs.num_of_writes,  
       CAST(vfs.io_stall_read_ms * 1.0 / NULLIF(vfs.num_of_reads, 0) AS decimal(10,2)) AS  
lat_lectura_ms,  
       CAST(vfs.io_stall_write_ms * 1.0 / NULLIF(vfs.num_of_writes, 0) AS decimal(10,2)) AS  
lat_escritura_ms,  
       CAST(vfs.num_of_bytes_read * 1.0 / NULLIF(vfs.num_of_reads, 0) / 1024 AS decimal(10,1)) AS  
kb_por_lectura  
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs  
JOIN sys.master_files mf ON mf.database_id = vfs.database_id AND mf.file_id = vfs.file_id  
ORDER BY vfs.io_stall DESC;  
-- Orientativo: log < 2-5 ms en escritura; datos < 10-20 ms en lectura  
-- Acumulado desde el arranque: tomar dos fotos y restar (delta)
```

02

Optimización y gestión avanzada de tempdb

Configuración de ficheros, contención de asignación y de metadatos, y las mejoras de 2019 y 2022

Quién usa tempdb (y por qué se satura)

- Objetos de usuario: tablas temporales #, ##, variables de tabla y cursores estáticos
- Objetos internos: spills de ordenaciones y hash, spools, tablas de trabajo de LOB y DBCC CHECKDB
- Version store: RCSI, SNAPSHOT, índices online, disparadores (tablas inserted/deleted) y réplicas legibles
- Miles de sesiones creando y borrando objetos pequeños: contención en páginas de asignación y metadatos
- Se recrea en cada arranque: su tamaño inicial configurado importa

En LegacyShop

El disparador de auditoría, el informe mensual con variables de tabla y los spills de la consulta de ventas por categoría golpean tempdb a la vez. Lo veremos en el laboratorio.

Dos tipos de contención en tempdb

Contención de asignación

- Páginas PFS (2:1:1), GAM (2:1:2) y SGAM (2:1:3)
- Espera PAGELATCH_UP / PAGELATCH_EX sobre esas páginas
- Solución clásica: varios ficheros de datos del mismo tamaño
- Una por núcleo hasta 8; luego de 4 en 4 si persiste
- 2022: concurrencia de páginas GAM/SGAM mejorada de serie

Contención de metadatos

- Tablas de sistema: sysschobjs, sysallocunits, syscolpars...
- PAGELATCH sobre páginas de objetos de sistema
- Más ficheros no lo resuelven
- 2019+: tempdb metadata optimizada para memoria
- Código: reutilizar tablas temporales en caché, evitar DDL tras crearlas

Configurar y vigilar tempdb

```
-- Ficheros actuales
SELECT name, physical_name, size*8/1024 AS mb, growth*8/1024 AS growth_mb
FROM tempdb.sys.database_files;

-- Ficheros del mismo tamaño y crecimiento fijo
ALTER DATABASE tempdb MODIFY FILE (NAME = tempdev, SIZE = 1024MB, FILEGROWTH = 512MB);
ALTER DATABASE tempdb ADD FILE (NAME = temp2, FILENAME = '/var/opt/mssql/data/temp2.ndf',
                                SIZE = 1024MB, FILEGROWTH = 512MB);

-- Metadatos en memoria (requiere reinicio)
ALTER SERVER CONFIGURATION SET MEMORY_OPTIMIZED TEMPDB_METADATA = ON;

-- ¿Qué página está en disputa ahora mismo?
SELECT r.session_id, r.wait_type, r.wait_resource, pi.object_id, pi.page_type_desc
FROM sys.dm_exec_requests r
CROSS APPLY sys.fn_PageResCracker(r.page_resource) prc
CROSS APPLY sys.dm_db_page_info(prc.db_id, prc.file_id, prc.page_id, 'LIMITED') pi
WHERE r.wait_type LIKE 'PAGELATCH%';
```

03

Infraestructura de almacenamiento

Buffer Pool Extension, acceso directo a almacenamiento persistente y alineación de bloques y páginas

Buffer Pool Extension (BPE)

- Extiende el buffer pool con un fichero en SSD para páginas limpias (nunca sucias)
- Útil cuando la RAM es limitada, la carga es de lectura aleatoria y los datos están en discos lentos
- Solo Windows, Standard y Enterprise; tamaño máximo 32 veces max server memory (4 veces en Standard)
- No aporta nada si los datos ya están en SSD/NVMe o si la carga es de escaneos secuenciales
- Hoy suele ser mejor añadir RAM: es una solución de nicho que conviene saber evaluar

Limitación del laboratorio

BPE no está disponible en SQL Server sobre Linux, así que no se puede activar en el contenedor. Veremos la configuración y las DMVs, y el laboratorio incluye la guía para probarlo en una VM Windows.

BPE: configuración y seguimiento (Windows)

```
-- Activar (Windows; SSD dedicado)
ALTER SERVER CONFIGURATION
SET BUFFER POOL EXTENSION ON
    (FILENAME = 'F:\SSDCACHE\LegacyShop.BPE', SIZE = 64 GB);

-- Estado y tamaño
SELECT path, state_description, current_size_in_kb / 1024 AS mb
FROM sys.dm_os_buffer_pool_extension_configuration;

-- ¿Cuántas páginas están en la extensión?
SELECT is_in_bpool_extension, COUNT(*) * 8 / 1024 AS mb
FROM sys.dm_os_buffer_descriptors
GROUP BY is_in_bpool_extension;

-- Desactivar
ALTER SERVER CONFIGURATION SET BUFFER POOL EXTENSION OFF;
```

Medir antes y después: Page Life Expectancy, lecturas físicas por segundo y latencia de lectura de los ficheros de datos.

Acceso directo al almacenamiento

1

Hybrid Buffer Pool (2019+)

Con memoria persistente (PMEM) en modo DAX, las páginas limpias se leen directamente del dispositivo sin copiarlas a DRAM.

2

Realidad del hardware

Intel discontinuó Optane, la principal PMEM. Funcionalidad vigente pero de adopción muy limitada en servidores nuevos.

3

Descarga de trabajo: Intel QAT

SQL Server 2022 puede descargar la compresión de backups en aceleradores Intel QAT, o usar su modo software.

4

Almacenamiento de objetos S3

BACKUP/RESTORE TO URL contra almacenamiento compatible con S3 y consulta de ficheros con PolyBase.

Alineación de bloques y páginas

- Offset de partición alineado a 1 MB (por defecto desde Windows Server 2008); en particiones antiguas, revisarlo
- NTFS/ReFS con unidad de asignación de 64 KB para datos, log y tempdb (coincide con la extensión)
- Linux: XFS o ext4; SQL Server recomienda XFS y opciones de montaje como noatime
- Sector físico mayor de 4 KB (algunos NVMe en Windows 11 / Server 2025): SQL Server no arranca o corrompe el log
- Inicialización instantánea de ficheros (IFI) y crecimientos fijos; en 2022 también el log hasta 64 MB

Comprobaciones rápidas

Windows: `fsutil fsinfo ntfsinfo X:` (Bytes per Cluster) y `fsutil fsinfo sectorinfo X:`. Linux: `xfs_info /ruta` y `cat /sys/block/<dev>/queue/physical_block_size`.

04

Diagnóstico de cuellos de botella con wait stats

PAGEIOLATCH, WRITELOG, ASYNC_IO_COMPLETION y cómo convertir esperas en decisiones

La metodología de esperas

- Toda tarea está ejecutándose (running), en cola de CPU (runnable) o esperando un recurso (suspended)
- `sys.dm_os_wait_stats` acumula desde el arranque: siempre trabajar con deltas
- Filtrar esperas benignas (`SLEEP_*`, `BROKER_*`, `XE_*`, `CHECKPOINT_QUEUE...`)
- `sys.dm_exec_session_wait_stats`: esperas por sesión, perfecto para aislar una consulta
- Las esperas indican síntomas, no causas: `PAGEIOLATCH` alto puede ser un índice que falta

Pensad como un médico

Una fiebre alta no dice qué infección hay, pero dice dónde mirar. Una espera dominante tampoco da la solución, pero descarta el 80 % de las hipótesis.

Las tres esperas de I/O del temario

1

PAGEIOLATCH_SH / _EX

Esperando que una página de datos llegue del disco al buffer pool. Mirar: consultas con más lecturas físicas, latencia de lectura por fichero, memoria y PLE.

2

WRITELOG

Esperando que el log se escriba en disco en el COMMIT. Mirar: latencia de escritura del log, commits fila a fila, VLFs, disco compartido con datos.

3

ASYNC_IO_COMPLETION

Esperando I/O asíncronos grandes: backups, crecimientos de ficheros sin IFI, algunas operaciones de datos. Mirar: backups en hora punta, autogrowth.

Delta de esperas en una ventana de tiempo

```
IF OBJECT_ID('tempdb..#w1') IS NOT NULL DROP TABLE #w1;
SELECT wait_type, waiting_tasks_count, wait_time_ms, signal_wait_time_ms
INTO #w1 FROM sys.dm_os_wait_stats;

WAITFOR DELAY '00:05:00';  -- ventana de observación (hora punta)

SELECT w2.wait_type,
       w2.wait_time_ms - w1.wait_time_ms AS espera_ms,
       w2.waiting_tasks_count - w1.waiting_tasks_count AS esperas,
       (w2.signal_wait_time_ms - w1.signal_wait_time_ms) AS signal_ms
FROM sys.dm_os_wait_stats w2 JOIN #w1 w1 ON w1.wait_type = w2.wait_type
WHERE w2.wait_time_ms - w1.wait_time_ms > 0
      AND w2.wait_type NOT LIKE 'SLEEP%' AND w2.wait_type NOT LIKE 'BROKER%'
      AND w2.wait_type NOT LIKE 'XE%' AND w2.wait_type NOT IN
        ('CHECKPOINT_QUEUE', 'LAZYWRITER_SLEEP', 'REQUEST_FOR_DEADLOCK_SEARCH',
         'SQLTRACE_INCREMENTAL_FLUSH_SLEEP', 'DIRTY_PAGE_POLL', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION')
ORDER BY espera_ms DESC;
```

signal_wait_time_ms es el tiempo en la cola de CPU tras obtener el recurso: si es alto en proporción, hay presión de CPU.

Lab 3 · tempdb: configuración y contención

- Auditar la configuración de tempdb del contenedor
- Provocar contención con carga concurrente de tablas temporales
- Identificar el tipo de página en disputa con dm_db_page_info
- Añadir ficheros, activar metadatos en memoria y volver a medir
- Comprobar la caché de tablas temporales en procedimientos

45 min

labs/Lab03_tempdb.md

Lab 4 · Cuellos de botella de I/O con wait stats

- Medir latencia por fichero con un delta de virtual file stats
- Provocar PAGEIOLATCH vaciando la caché y midiendo el efecto de un índice
- Provocar WRITELOG con commits fila a fila frente a lotes y delayed durability
- Provocar ASYNC_IO_COMPLETION con un backup y con crecimiento de ficheros
- Redactar el diagnóstico de cada escenario en tres líneas

50 min

labs/Lab04_Waits_IO.md

Cierre de la sesión 2

- La latencia por fichero y las esperas son el idioma del disco: siempre con deltas
- tempdb: ficheros iguales, tamaño inicial adecuado y metadatos en memoria si hay contención de sistema
- BPE y memoria persistente son soluciones de nicho; la RAM y los índices suelen ganar
- Alineación, IFI y crecimientos fijos evitan problemas silenciosos
- Próxima sesión: leer planes de ejecución complejos como un profesional

Para casa

Ejecutad la consulta de latencia por fichero en un entorno real y clasificad cada fichero de log por encima o por debajo de 5 ms de escritura media.